

AI Agent Platform

Architecture Design and Technical Documentation

Contents

1	Architecture Design	3
1.1	What the platform is	4
1.2	Architecture Design	4
1.3	What each component is responsible for	5
1.3.1	Frontend (web client)	6
1.3.2	Agent Management	6
1.3.3	Agent Execution (the orchestrator)	6
1.3.4	LLM Provider	6
1.3.5	Tool Provider	7
1.3.6	Run History	7
1.3.7	Session Management	7
1.3.8	Trigger Management and the Trigger Scheduler	7
1.3.9	Persistence	8
1.3.10	The composition root	8
1.4	How a request flows through the system	8
1.4.1	A chat message	8
1.4.2	A trigger firing	9
1.5	Design decisions	10
1.6	Deployment architecture	11
1.6.1	Mapping to a cloud-native target	12
1.7	Tradeoffs, limitations, and risks	13
2	Technical Documentations	14
2.1	Overall architecture	15
2.1.1	System at a glance	15
2.1.2	Agent execution	16
2.1.3	Conversations and scheduled execution	18
2.1.4	Client and API boundary	19
2.2	API reference	19
2.2.1	Error envelope	20
2.2.2	Health	20
2.2.3	Agents	20
2.2.4	Tools and models	21

2.2.5	Chat and execution	22
2.2.5.1	Request body	22
2.2.5.2	Response shapes	22
2.2.5.3	Streaming events	22
2.2.5.4	The deterministic failure path	23
2.2.6	Sessions	23
2.2.7	Triggers	24
2.2.8	Runs	25
2.3	Data model	26
2.3.1	Entities and relationships	26
2.3.2	Deletion behavior	26
2.3.3	Physical schema	27
2.3.4	Startup schema management and backfill	27
2.4	Extension interfaces	28
2.4.1	Tool boundary	28
2.4.2	LLM provider boundary	29
2.5	Deployment guide	30
2.5.1	Run the container stack	30
2.5.2	Configuration reference	31
2.5.3	Run against live Gemini	32
2.5.4	Common deployment problems	33
2.5.5	Run locally without containers	33
2.6	Developer guide	33
2.6.1	Add a tool	33
2.6.2	Add an LLM provider	34
2.6.3	Add a storage backend	35
2.6.4	Verification	35
2.6.5	Extension checklist	35
2.7	Known limitations	36

Chapter 1

Architecture Design

This document explains what the platform is, why it is designed this way, and how requests move through the system.

1.1 What the platform is

The AI Agent Platform lets users create reusable custom agents. Each agent has a name, a selected LLM model, a system prompt, and a set of tools. Users can run agents in two ways: manually through chat or automatically on a schedule. Every run is saved, including the user request, the model's actions, the tools it used, and the final response. This lets users review what an agent did and how it produced its result.

An agent can be started in two different ways, and the system handles them as two separate types of execution:

- **A person**, through the browser, who creates and edits agents, chats with them, manages chat threads, and manages triggers.
- **The trigger scheduler**, a clock inside the system itself, with no browser session and no user behind it, that fires an agent when its schedule comes due.

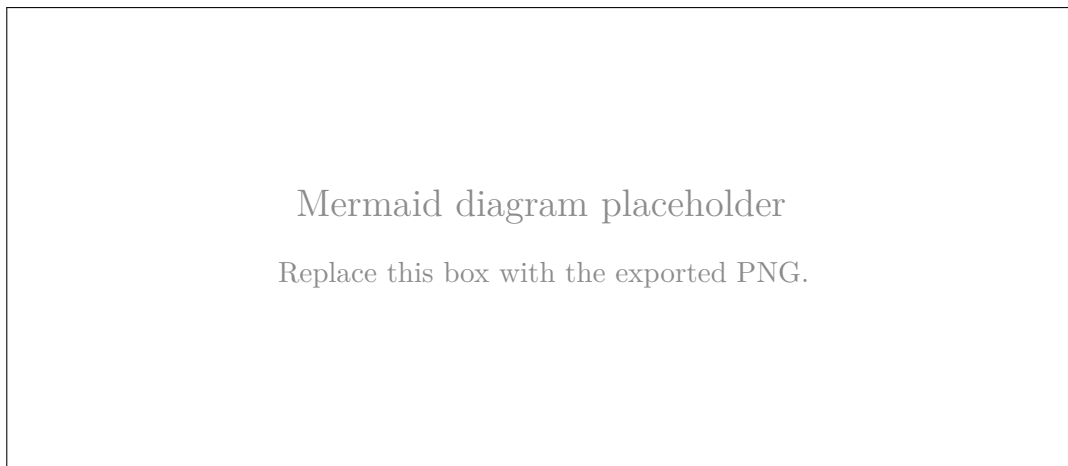


Figure 1: Platform actors and use cases

- **Configuring an agent is part of managing it.** The system prompt and tool list are stored together with the agent's name, model, and status. This keeps the agent's settings in one place and avoids having separate configuration screens or endpoints.
- **Every execution is saved in run history.** Whether an agent is started from chat, triggered manually, or triggered on a schedule, the system always creates a run record. This means users can later review what happened during any execution.

Implementation. The one human actor covers every route the frontend exposes — agent management, chat, and trigger management. The scheduler actor is a background loop with no HTTP request or session behind it (section 4.2). There is no login or role separation in this proof of concept, so there is exactly one kind of human actor to model.

1.2 Architecture Design

The system has two main parts: a browser app that users interact with, and a backend that manages agents, runs tasks, and stores history. The backend is split into smaller components,

with each component handling one specific job and communicating with the others through clear interfaces.

Each component depends on an interface instead of depending on one specific implementation. A component can ask for an LLM or a storage service through the port, and the system gives it the version that is currently configured. This means the same component can use an in-memory store and a mock model during development, or Postgres and Gemini in production, without changing its own code.



Figure 2: High-level platform architecture

Agent Execution is the orchestrator because it connects to most parts of the system, including agent settings, tools, the LLM, run history, and sessions. Its job is to coordinate all of these parts for one agent run. Other components only depend on the few services they actually need.

Design decision — use ports instead of specific providers. Agent Execution does not directly depend on one LLM client or one database. Instead, it uses interfaces, so the LLM or storage provider can be changed without changing the business logic.

Implementation. The backend is a single FastAPI service, organized as modules under `app/modules/`: `agents` (Agent Management), `execution` (Agent Execution), `llm` (LLM Provider), `tools` (Tool Provider), `runs` (Run History), `sessions` (Session Management), `triggers` (Trigger Management and the scheduler). Each port is a Python `Protocol`; the concrete implementation behind it is chosen in exactly one place, `app/container.py`, based on environment configuration — never wired case-by-case inside a service. The frontend is a React single-page app whose only channel to the backend is `fetch` calls to `/api/*`, funneled through one client module so no component or hook talks to the network directly.

Two ports have more than one implementation available, selected by an environment setting:

Port	Implementations	Selected by
LLM Provider	Mock model or Gemini through Google ADK	<code>LLM_PROVIDER=mock</code> <code>adk_gemini</code>
Storage	In-memory storage or PostgreSQL	<code>STORE_BACKEND=memory</code> <code>postgres</code>
Tool Provider	Four built-in tools	Fixed for this deliverable

1.3 What each component is responsible for

Each component below is explained by what it does, why it has that job, how it connects to the rest of the system, and where it is implemented.

1.3.1 Frontend (web client)

The browser application is what the user interacts with. It provides pages for managing agents, chatting, managing triggers, and viewing run history.

All user actions are sent to the backend as API requests. The frontend keeps all API communication in one place, so the rest of the frontend does not need to know the API details.

The frontend never talks directly to the LLM or tools. It always sends requests to the backend, and the backend decides what happens and stores the result.

Implementation. `AgentPlatform-FrontEnd/client`, with `src/lib/api-client.ts` as the single module that calls `fetch`.

1.3.2 Agent Management

Create, read, update, and delete agents; validate which model and which tools an agent is allowed to use.

An agent is a piece of configuration, not a running process — it needs a place that owns its lifecycle and enforces that its fields (model, tools, prompt) are always internally consistent, independent of whether that agent is ever executed.

Deleting an agent also deletes its chat sessions and triggers because they cannot work without the agent. However, its run history is kept because it records actions that already happened and should not be erased.

Implementation. `app/modules/agents`. It is handed the set of valid tool ids and model ids rather than importing the catalogs, so it can tell whether a tool exists without knowing what any tool does. That boundary is not a convention — a structural test fails the build if this module ever imports tool or model implementation code.

1.3.3 Agent Execution (the orchestrator)

Everything involved in running one agent turn: load the agent's configuration, work out which tools it's allowed to use, send the request to the model, handle any tool calls the model makes along the way, and produce a finished result plus a permanent record of it.

This is deliberately the one place in the system that knows how to run an agent end to end, so that the two very different things that can trigger a run — a **chat message** and a **scheduled trigger** — can share every step of that process except the parts that are genuinely different between them.

It coordinates the tools and the LLM through their interfaces, without needing to know which specific implementations are being used. It also leaves session handling to Session Management, because scheduled runs do not have a chat session.

Implementation. `app/modules/execution`. Three entry points sit on one shared core: `send_message` and `stream_message` for chat, which differ only in how the answer is delivered, and `run_trigger` for scheduled and manual firing.

1.3.4 LLM Provider

The LLM Provider defines what every model backend must support: listing available models and running one agent turn from the agent settings and one user message.

Agent Execution uses the same interface whether the system is using a mock model or a real model such as Gemini. Each provider returns the same simple events: tool call started, tool call finished, text received, and turn finished. This means the rest of the system does not

need to understand each provider's own API.

The port does not include operations like summarize or create title. A new chat title is made by shortening the first message, so it does not need a model call. The port also receives only one message at a time, so previous conversation messages are not sent back to the model.

Implementation. `app/modules/llm`, with the mock and ADK/Gemini implementations under `providers/`. Which one is active is decided only in `app/container.py`.

1.3.5 Tool Provider

The contract a tool must satisfy, the registry that knows the full list of available tools, and the built-in tools themselves.

A tool is meant to be simple to add and simple to reason about in isolation: given arguments, produce a result. Keeping that contract free of any awareness of agents, runs, or the model is what makes a new tool a self-contained addition rather than a change that ripples elsewhere.

A tool returns an error result for normal problems, such as invalid input or a failed external request, instead of crashing. If a more serious error happens, the system catches it, records the tool call as failed, and sends the error back to the model. This means one failed tool does not stop the whole agent run.

Implementation. `app/modules/tools`, with implementations under `impls/` and the `ToolRegistry` as the only module that knows the concrete list.

1.3.6 Run History

The run history is the permanent record of every agent execution. Each run stores the user request, the model response, the tools that were used, and how the run ended.

The record is kept even if the related agent, session, or trigger is later changed or deleted. Each run also saves the agent's name, model, and system prompt as they were when the run happened, so the history always shows what actually happened at that time.

Implementation. `app/modules/runs`.

1.3.7 Session Management

Chat thread identity: creating a new thread on its first message, giving it a title, and letting a person rename or delete it later.

A chat conversation needs an identity independent of any single message in it, so that a person can return to it, and so that run history can group runs by conversation.

A session doesn't exist until the first message of a thread is sent — there's no "create empty chat" action — and it's titled automatically from that first message rather than requiring a person to name it up front. Deleting a session removes the runs that belong to it, because the runs *are* the conversation, not a description of one.

Implementation. `app/modules/sessions`. A triggered run has no session at all, by design (section 4.2).

1.3.8 Trigger Management and the Trigger Scheduler

CRUD for triggers, the logic that computes when a trigger is next due, and a background process that watches for due triggers and fires them.

This is the platform's only way to run an agent without a person present. Separating "what a trigger is" from "the clock that fires it" keeps the REST surface for managing triggers usable

even in a mode where firing itself is turned off.

The scheduler does not run inside a request/response cycle at all — it's a loop that polls on an interval, entirely independent of any browser request.

Implementation. `app/modules/triggers`, with the scheduler in `scheduler.py` running as a background task for the life of the process.

1.3.9 Persistence

Storage for each entity — agents, runs, sessions, triggers — behind its own port.

Nothing above this layer should need to know or care whether "storage" means a Python dictionary in memory or a Postgres table; both need to satisfy exactly the same contract.

Two implementations exist per port today — in-memory (fast, disposable, used for local development and tests) and Postgres (durable) — chosen once per process, never per call.

Implementation. `repositories/memory.py` and `repositories/postgres.py` under each module.

1.3.10 The composition root

The composition root is the one place where the system chooses which implementation to use for each port and gives it to the components that need it.

Keeping these choices in one place makes it clear which LLM provider and storage backend are active. It also ensures that components use ports instead of directly depending on specific implementations.

Services and routers receive the components they need through dependency injection, such as constructor arguments or FastAPI dependencies, instead of importing implementation modules directly.

Implementation. `app/container.py`. This module does not handle requests or contain business logic. Its only job is to connect components together.

1.4 How a request flows through the system

1.4.1 A chat message

This is the platform's core demonstrable flow: a person sends a message, the system decides whether a tool is needed, sends the context to the model, and returns a complete answer along with a permanent record of everything that happened.

Three parts of this flow are important:

- **The turn is streamed, but the run record is saved only when it is complete.** The browser receives tool calls and model text as they happen, so the user can see progress during a long run. The final run record is saved in full before the last event is sent. If the stream ends before that, the client treats the run as failed instead of showing a partial result.
- **The failure path can be tested.** A message that matches a simple pattern can intentionally cause a tool call to fail, unless the request is marked as a retry. This makes it easy to test how errors and retries behave without changing code or system settings.
- **Session handling stays outside the shared execution logic.** Creating, checking, and naming a session only apply to chat. Keeping this logic separate allows scheduled triggers to use the same execution flow without creating an unnecessary session.

Implementation. `execution/router.py` uses `ExecutionService.send_message` for the normal JSON route and `stream_message` for the streaming route. Both use the same provider event

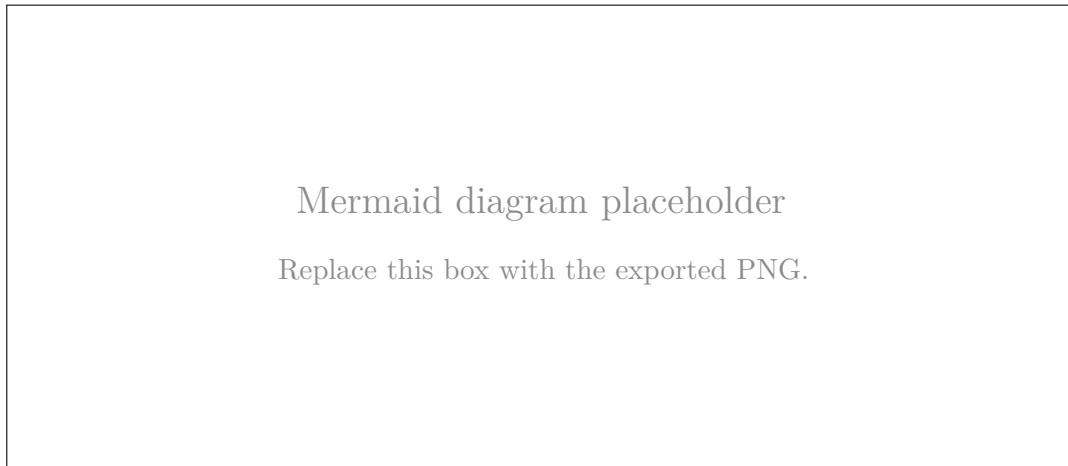


Figure 3: Chat message execution sequence

stream, represented as `AsyncIterator[RunEvent]`. `event_translator.py` converts these events into the final message and run record. The streaming route also sends each event to the browser as newline-delimited JSON while the run is happening.

1.4.2 A trigger firing

Triggers are the platform's second and only other way into agent execution, and the only one where no request initiates anything — a background process does.

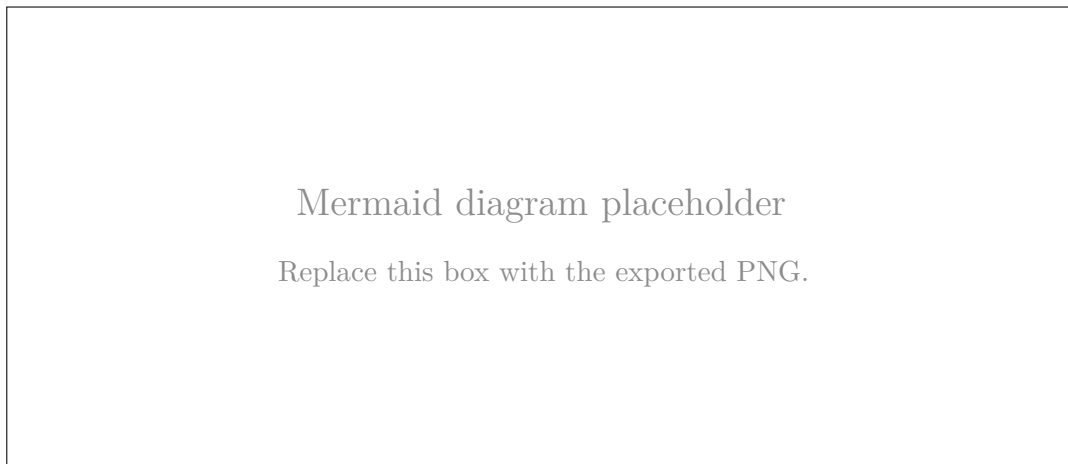


Figure 4: Scheduled trigger execution sequence

Several choices in this flow only make sense once you see the constraint they're solving for:

- **A missed period of downtime becomes one catch-up firing, not a replay of every firing that was missed.** The next scheduled time is always computed from *now*, not from the last time it should have fired. This trades exact historical replay for a simpler, bounded recovery: however long the scheduler was down, it fires each trigger at most once when it comes back.
- **Claiming a trigger is a compare-and-swap, not an unconditional update.** The claim moves the trigger's next due time forward only if it still holds the value this tick read, which is also why rescheduling happens *before* the run rather than after it: a trigger stops being due the moment it is claimed, so nothing can pick it up again while it is still executing.

This is what makes it safe to ever run more than one instance of the scheduler — the second instance's claim on an already-claimed trigger fails harmlessly instead of firing the agent twice.

- **One trigger failing does not stop the rest.** A firing that raises is caught, logged, and recorded as an error status on that trigger; the tick continues to the next one.
- **A triggered run is intentionally invisible in chat.** It has no session, so it never appears in the chat sidebar — it's reachable only through that trigger's own activity log. This keeps "conversations a person is having" and "things happening automatically in the background" from ever being visually confused with each other.

Implementation. `triggers/scheduler.py`, running as a lifespan background task, calling `ExecutionService.run_trigger` — the same orchestrator core used by chat, minus session handling.

1.5 Design decisions

The previous sections mentioned several choices in passing. This section states the significant ones explicitly: what was chosen, what problem it solves, what the realistic alternative was, and what it costs.

Ports between orchestration and every external dependency. *Chosen:* Agent Execution and its neighbors depend on interfaces (LLM Provider, Tool Provider, Persistence), never on a concrete implementation. *Problem solved:* the platform must be able to run offline and deterministically (mock model, in-memory store) for development and testing, and against real infrastructure (Gemini, Postgres) for actual use, without the orchestration logic caring which. *Alternative:* import a specific SDK and a specific database driver directly into the orchestrator. *Tradeoff:* an added layer of indirection, and a discipline that has to be maintained — a new provider must be shaped to fit the existing port rather than allowed to leak its own quirks upward.

A deterministic mock is the default LLM provider. *Chosen:* the platform boots and demonstrates a full execution flow — including the tool-calling and failure paths — with no external API key and no network dependency, by default. *Problem solved:* development, automated tests, and a live demonstration all need to be repeatable and free of external cost or flakiness. *Alternative:* require a real model API key from the start. *Tradeoff:* the mock's behavior is scripted rather than genuinely intelligent, so it can only prove the *plumbing* works, not that a real model will behave well inside it — switching to the live provider is one environment variable, and the normalized event contract (section 3) is what guarantees that switch doesn't require any code change.

Streaming is an extra delivery route, not a second execution path. *Chosen:* the chat surface exposes two endpoints over identical work — one returning the finished message as JSON, one streaming the same turn as newline-delimited events and ending with that same finished message. The browser uses the streaming route so a tool-using turn shows its work as it happens. *Problem solved:* an agent turn that calls two tools takes seconds, and a UI that shows nothing until it ends reads as broken; but a system whose only delivery mode is a live connection has to answer awkward questions about partially recorded history. *Alternative:* pick one — stay synchronous everywhere and accept the wait, or go streaming-only and give up the simple request/response contract that the tests and any scripted client rely on. *Tradeoff:* two routes to keep in step, and a class of failure that only exists once headers are sent. Both costs are contained deliberately: the routes share one code path and differ only in what the router does with each event, and a failure after headers arrives as an in-band error event because a status code is no longer available. The run is written before the finishing event goes out, so history is never partial regardless of which route was used or how the connection ended.

Run history stores a snapshot, not a reference. *Chosen:* each run copies the agent's name, model, and system prompt as they were at the moment it ran, rather than looking them up live from the current agent record. *Problem solved:* an audit trail that changes when the thing it's describing changes isn't an audit trail. Editing an agent tomorrow must not silently rewrite what it appears to have done yesterday. *Alternative:* store a reference (a foreign key) to the agent and join against its current fields when displaying history. *Tradeoff:* deliberate duplication of data, and a small amount of coupling — a future field added to an agent that should also appear in history has to be added to the snapshot explicitly, since nothing keeps them in sync automatically.

Deletion cascades are asymmetric, and encoded as application logic rather than a single blanket rule. *Chosen:* deleting an agent removes its sessions and triggers but keeps its runs; deleting a session removes its runs; deleting a trigger keeps its runs. *Problem solved:* each of those relationships means something different — a session *is* its runs, while an agent or a trigger merely *produced* runs that should outlive it as history. *Alternative:* a single uniform cascade-delete rule at the database level. *Tradeoff:* the deletion behavior lives in each owning component rather than being visible in one place as a schema constraint, so it has to be read as code, not inferred from a diagram of foreign keys — the tradeoff is precision of business meaning against that single source of truth.

The trigger scheduler polls; it does not queue. *Chosen:* a background loop wakes up on a fixed interval and asks storage which triggers are due, rather than a message queue or a dedicated external job scheduler pushing work in. *Problem solved:* the platform gets scheduled execution without introducing a new infrastructure dependency (a queue, a separate scheduler service) for a proof of concept that needs to be simple to run locally and in a small container deployment. *Alternative:* a message queue or a managed scheduler product driving execution. *Tradeoff:* firing precision is bounded by the poll interval, not exact to the second, and the current model implicitly assumes one active scheduler instance — running several safely (for horizontal scaling) would need the compare-and-swap claim in section 4.2 to be joined by a dedicated job runner, which section 7 covers as an open risk.

The container deployment is Docker Compose, mapped onto managed cloud services rather than built on them from the start. *Chosen:* three containers (web, api, database) orchestrated by Compose satisfy this deliverable's proof-of-concept deployment requirement directly. *Problem solved:* a reviewer or developer needs to bring up the entire stack with one command and no cloud account. *Alternative:* build directly on a cloud-native target (Cloud Run, Cloud SQL) from day one. *Tradeoff:* Compose does not exercise the operational realities of a managed, multi-instance deployment — see section 6 and section 7 for what that mapping would actually require.

1.6 Deployment architecture

The platform runs the same way regardless of environment — the only thing that changes is which concrete provider and storage backend are configured, per the ports pattern in section 2.

Local development runs as two plain processes on the developer's machine: the backend service on port 4000, and the frontend's dev server on port 5173, which forwards `/api` requests to it. No container is involved.

The container deployment — the artifact for this deliverable — brings up three containers together:

Publishing only the web container keeps the browser same-origin with the API — every request goes to the same host and port the page was served from, so there is no cross-origin configuration to get right or wrong in this deployment. The API itself is never reachable from outside the container network. Startup order is enforced by health checks rather than a fixed

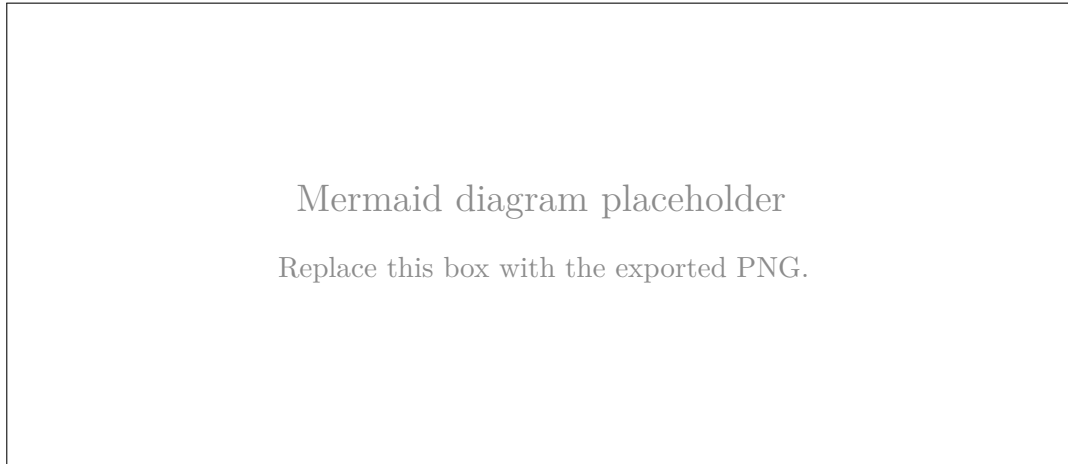


Figure 5: Container deployment architecture

delay: the database must report healthy before the API starts, and the API must report healthy before the web container begins serving — so a slow database on a loaded machine delays startup instead of producing a race that intermittently fails.

Configuration is entirely environment-driven, through a file that is never committed to version control — only its template is, and a fresh clone does not need even that, because every setting has a default in the compose file. The container stack always runs against real PostgreSQL; what the default configuration changes is the *model*, which starts as the offline mock. So the stack comes up and demonstrates a complete execution flow — tool calls, traces, durable history that survives a restart — with no external credentials and no cost. Pointing it at the live model provider is a one-variable change plus an API key, with no code change required, which is the entire payoff of the ports decision in section 5.

Implementation. `AgentPlatform-BackEnd/deployment`, driven by `docker compose up --build`; configuration in `deployment/.env`, copied from the tracked `.env.example` and needed only to change a default. The full variable table is in `docs/technical-documentation.md` section 5.

1.6.1 Mapping to a cloud-native target

Each container is already built independently with no host-specific assumptions, so mapping this deployment onto managed cloud infrastructure is a direct substitution, not a redesign:

Container	Cloud equivalent
Web (reverse proxy + static client)	A managed container run service, or static hosting plus a CDN for the client bundle
API (backend service)	A managed container run service — stateless and scales on request volume, with one caveat: the trigger scheduler's background-loop model (section 4.2) assumes a single active instance, so scaling the API to multiple instances would need the scheduler moved to a dedicated, singleton job runner or a managed scheduler product
Database	A managed PostgreSQL service
Environment configuration file	A managed secrets service, injected as environment variables at deploy time

This mapping is not implemented in this proof of concept — Docker Compose is the delivered artifact — but nothing about the current design blocks it.

1.7 Tradeoffs, limitations, and risks

This section collects what the design choices above cost, in one place, rather than leaving each cost implicit in its own section.

- **The trigger scheduler assumes a single active instance.** The compare-and-swap claim (section 4.2) prevents two instances from firing the same trigger twice, but nothing today moves scheduling itself off the API process. Scaling the API horizontally without also addressing this would either need every instance's scheduler loop to keep racing harmlessly (wasteful, and the currently expected mode) or the scheduler pulled out into its own singleton process — this is the clearest structural risk in the design if the platform grows past a single instance.
- **There is no authentication or authorization.** One implicit user is assumed throughout; there is no login, no per-user data isolation, and no permission model. This is appropriate for a proof of concept and actively wrong for anything handling more than one person's data — it is the first gap to close before any real deployment.
- **A chat thread is a record of a conversation, not a memory of one.** Each turn sends the model the agent's system prompt and the current message only; earlier turns in the same session are not replayed. A session groups runs so a person can return to a thread and read it back, and it is genuinely durable — but the agent itself does not remember what was said three messages ago, so a follow-up like "and the other one?" will not resolve. Adding memory is a change to what the execution core sends the provider, not to the shape of the system: the thread is already stored, already ordered, and already retrievable. It is the most visible functional gap between this platform and a product-grade assistant.
- **The default configuration is demonstration-shaped, not production-shaped.** The mock model and default storage settings are what make the platform boot with zero external dependencies, which is a deliberate and correct choice for this deliverable — but it also means the failure modes, latency, and cost of a real model are invisible until someone deliberately switches the provider. A team relying on this platform in earnest needs to exercise the live-provider path, not just the default one, before trusting its behavior.
- **Streaming adds a failure mode that has no status code.** Once the response has started the status is already 200, so a provider failure mid-turn reaches the browser as an error event inside the stream rather than as a 502 (section 5). Every client therefore has to understand two ways a request can fail, and one of them is only visible to a client that reads the body to the end. A dropped connection is also invisible to the server as anything other than a stream nobody finished reading.
- **Snapshotting couples run history's shape to the agent's shape.** Any new field added to an agent that should also be preserved in history has to be added to the snapshot deliberately; nothing enforces that they stay in sync automatically, and this is a place a future change could quietly go stale.
- **A single database instance is a single point of failure.** The current deployment includes no replication, failover, or backup strategy beyond what the underlying managed service would provide if adopted — this proof of concept does not address durability beyond "data survives a container restart."

Chapter 2

Technical Documentations

This document is Deliverable 2 from `AI Agent Platform.pdf`. Deliverable 1, `docs/architecture-design.md`, explains in plain language what the platform is and why it is shaped the way it is. This document is the engineering reference behind that explanation: the component boundaries as they are actually implemented, the HTTP contract field by field, the persistent data model, the two extension interfaces, and the commands to deploy and develop against it.

Everything here was checked against the code in this repository and against a running instance of the service, not against the design notes that preceded it. Where an older plan or spec disagrees with the running system, the running system is what is documented.

2.1 Overall architecture

2.1.1 System at a glance

The platform lets a person define agents and run them two ways: interactively through chat, or unattended on a schedule. An agent is a name, an icon, a description, a model, a system prompt, and a list of allowed tools. Every completed turn — chat or scheduled — writes one auditable run record holding the question, the answer, the ordered tool calls with their arguments, results, and durations, the latency, the status, and a snapshot of the agent configuration that produced it.

The backend separates agent configuration, execution orchestration, model integration, tool integration, and persistence, because those five concerns change for different reasons and on different schedules. A new model provider should not touch agent management. A new tool should not need a new route. Changing the storage backend should not alter a business rule. The same boundaries are what make the default configuration — a deterministic mock model over an in-memory store — a complete, exercisable system that needs no credentials and no database.

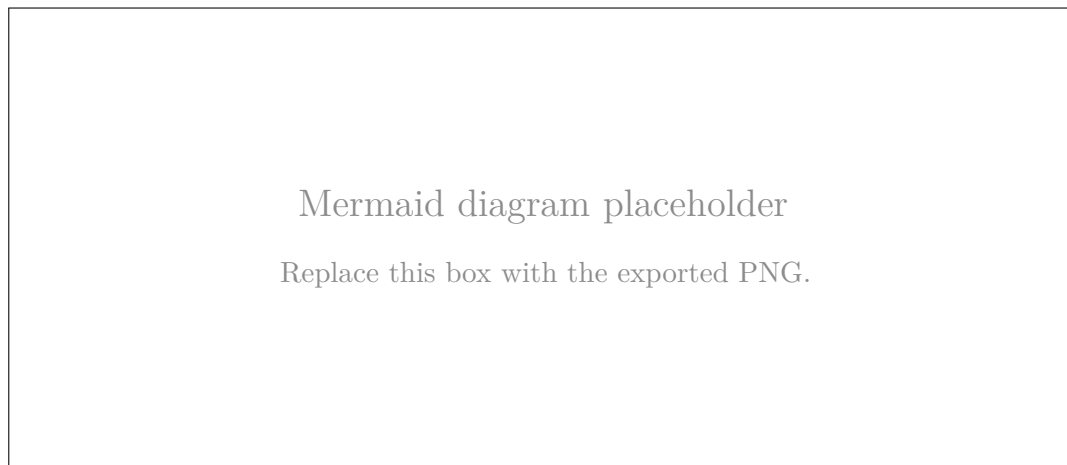


Figure 6: Overall system architecture

Design decisions.

- **One deployable service.** The whole backend is a single FastAPI application. This keeps the proof of concept operationally simple, at the cost of not being able to scale or deploy a component independently.
- **Business components depend on interfaces, not implementations.** Every collaborator that could plausibly be swapped — storage, the model, the tool catalog — is reached through a `Python Protocol`. That is what allows in-memory and PostgreSQL stores, a deterministic and a live model provider, and isolated tests, with no change to the code that uses them.

- **Chat and triggers share one execution core.** Scheduled execution is not a second engine with its own tool handling, failure handling, or audit behavior. The two entry points differ only in what surrounds the shared core.
- **Provider events are normalized at the boundary.** Nothing above `LLMProvider` sees a vendor event type. The price is that each adapter must translate its SDK's native event model into the platform's four-event vocabulary, preserving ordering, timing, and result shape.
- **The composition root is one file.** `app/container.py` is the only module that picks a concrete repository or provider, and `app/config.py` is the only module that reads the environment. If either choice could be made in a second place, "which provider is running" would stop being a single fact about the process.

Implementation mapping.

Conceptual component	Where it lives
Web client	React 19 and TypeScript on Vite, in <code>AgentPlatform-FrontEnd/client</code>
HTTP API	FastAPI application built by <code>create_app()</code> in <code>AgentPlatform-BackEnd/server/app/main.py</code>
Agent Management	<code>app/modules/agents</code> , business rules in <code>AgentService</code>
Agent Execution	<code>ExecutionService</code> and <code>event_translator.py</code> in <code>app/modules/execution</code>
LLM Provider	<code>LLMProvider</code> in <code>app/modules/llm/provider.py</code> ; implementations <code>MockLLMProvider</code> and <code>AdkGeminiProvider</code>
Tool Provider	<code>Tool</code> , <code>ToolSchema</code> , <code>ToolRegistry</code> , and the four tools under <code>app/modules/tools</code>
Run History	<code>app/modules/runs</code>
Session Management	<code>app/modules/sessions</code>
Trigger Management and scheduler	<code>app/modules/triggers</code> , split between <code>TriggerService</code> and <code>TriggerScheduler</code>
Persistence	A repository Protocol per module, with memory and PostgreSQL implementations under each module's <code>repositories</code> /; SQL in <code>app/core/schema.sql</code>
Composition root	<code>app/container.py</code>

The backend is organized as vertical slices under `app/modules/`, not as global `controllers` /, `services` /, and `models` / folders. Everything genuinely cross-cutting — the error envelope, the body-size limiter, the Express-compatible JSON parser, the connection pool, identifier generation, the injectable clock, and camelCase wire conversion — lives in `app/core/`.

`tests/test_structure.py` enforces the boundaries mechanically. It imports every module in the tree, which is why the optional Google ADK import is deferred into the functions that use it, and it asserts that nothing under `app/modules/agents/` imports the tool or model catalogs.

2.1.2 Agent execution

`ExecutionService` coordinates one agent turn. It loads the agent's current configuration, resolves the tool ids stored on it into callable tools, builds a provider-neutral run specification, consumes the provider's event stream, folds those events into a user-facing message and a run record, and appends the run to history.

This is the one component that depends on nearly everything else, and that is its job. The alternative — spreading the same coordination across the router, the trigger scheduler, and each provider adapter — would give three places the chance to disagree about what a turn is.

How a turn is assembled.

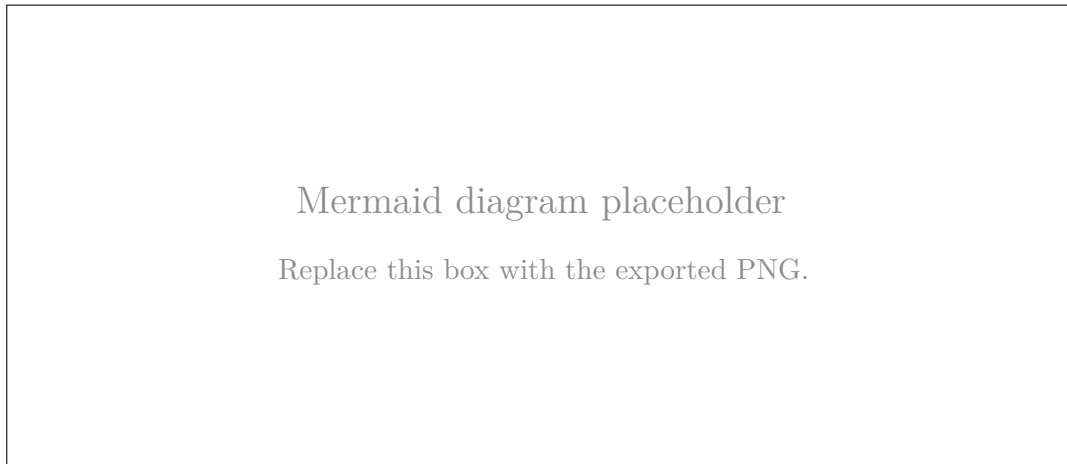


Figure 7: Agent execution flow

1. A chat request or a trigger firing names an agent and supplies a message.
2. The agent is loaded; an unknown id is a 404 before anything else happens.
3. `ToolRegistry.resolve()` turns the stored `toolIds` into tools, in the stored order.
4. A `RunSpec` carries agent identity, display name, model, system prompt, resolved tools, the message, the retry flag, and the optional session id, trigger id, and timezone.
5. The provider runs one complete agent turn — including any model-directed tool loop — and emits `ToolCallStarted`, `ToolCallFinished`, `TextDelta`, and finally `TurnFinished`.
6. `EventAccumulator` pairs each start with its finish by `call_id`, preserves invocation order, and derives status: a call is `error` if it reported one, and the turn is `error` if any call failed. A start with no matching finish is recorded as a failed call carrying the message `The provider ended the turn before this call reported a result.`
7. The answer is `TurnFinished.text`, falling back to the concatenated text deltas when the provider sends no final text.
8. Tool arguments and results larger than `LOG_PAYLOAD_MAX_BYTES` are replaced in the **stored row only** with `{"truncated": true, "bytes": N, "preview": "..."}.` The HTTP response is never altered, so the trace on screen always shows what the tool actually returned.
9. The run is appended. Nothing ever updates an existing run.

Design decisions.

- **The provider port represents one agent turn, not one model completion.** The tool-calling loop belongs to the model SDK. Modeling a single completion would have forced that loop up into `ExecutionService` and made every adapter reimplement it differently.
- **A run snapshots agentName, model, and systemPrompt.** Agents are mutable. Resolving those through the current agent at read time would let an edit rewrite the historical explanation of a past run.
- **Tool failures are data; provider failures abort the turn.** A failed tool call is stored and returned inside a completed turn, because the model saw it and answered around it. A provider failure raises `ProviderError`, answers 502, and appends **no** run, because no complete event sequence exists to record.
- **Upstream error text never reaches the response.** `llm/failures.py` classifies the raw exception into one of five short sentences — quota, credentials, timeout, unavailable, unknown — and the vendor's own message, which carries billing URLs and sometimes the prompt, goes to the log instead.
- **Chat-specific work stays outside the shared core.** Session creation, ownership checks, titling, and activity updates live in the chat entry points. `run_trigger()` deliberately creates

no session, so a firing can never appear in the chat sidebar.

Implementation mapping. `ExecutionService._execute()` is the shared core; `send_message()` and `stream_message()` add the chat concerns around it; `run_trigger()` supplies a trigger id and a timezone and omits the session. `event_translator.py` holds `EventAccumulator` and `translate()` and knows nothing about any specific provider.

2.1.3 Conversations and scheduled execution

A session is the identity and title of a human conversation. A trigger is a stored message plus a schedule. Both produce runs, but only chat produces sessions.

The first message of a chat creates its session and titles it by deterministic truncation of that message — no model call, so the first turn costs nothing extra and the title is stable and offline. Later messages must send the returned `session.id`. The session must exist, or the request is a 404, and it must belong to the agent named in the URL, or the request is a 400. Both checks run before the provider call and before the run is appended, so a rejected send leaves nothing behind.

The scheduler is one `asyncio` task started by the FastAPI lifespan.

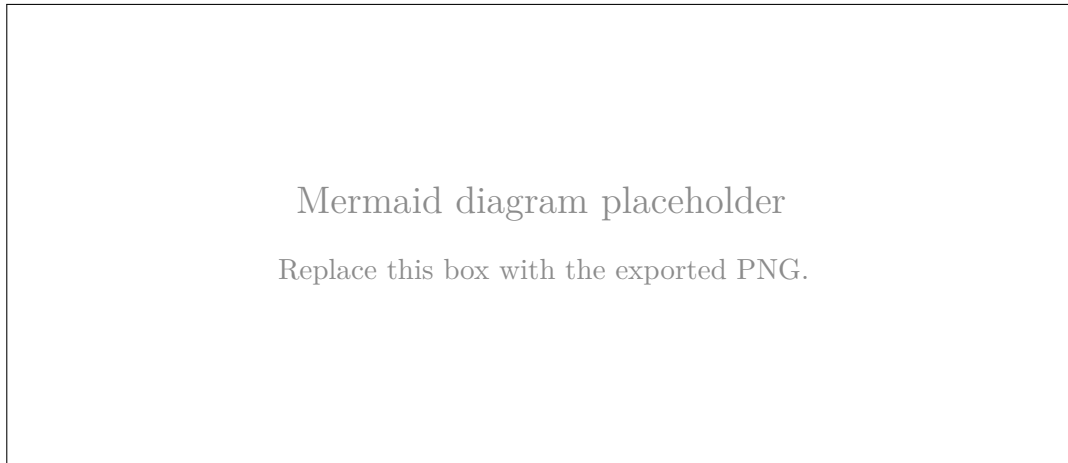


Figure 8: Trigger scheduler flow

Design decisions.

- **Missed firings collapse instead of replaying.** The next fire time is computed from the current instant, not from the stale `next_run_at`. After an outage a trigger fires once and resumes from now, rather than emitting a backlog of catch-up runs.
- **Claiming is a compare-and-swap.** `claim()` moves `next_run_at` forward only if it still holds the value the tick read. A process that loses the race simply does not fire; a lost claim is not an error.
- **Firings are awaited sequentially.** With a single scheduler task this is also what stops a trigger from overlapping its own previous firing when a run outlasts its interval.
- **The clock is separable from the engine.** `tick()` is the whole engine and the loop around it is a wrapper, so moving scheduling into a dedicated worker later is a change of entry point, not a redesign.
- **`TRIGGERS_ENABLED=false` stops the clock only.** Every trigger route keeps working, including fire-now, which is deliberately independent of both the enabled flag and the scheduler.

Implementation mapping. Session lifecycle is split between `ExecutionService._prepare_message()` (create, validate, title) and `SessionService` (list, rename, delete). Trigger

CRUD and fire-now are `TriggerService`; polling and scheduled firing are `TriggerScheduler.tick()` and `run_scheduler_loop()`. Next-fire computation is a pure function in `triggers/schedule.py` with no clock and no I/O, which is what makes the only subtle logic in the feature readable on its own.

2.1.4 Client and API boundary

The browser application is a separate single-page app that reaches the backend only through JSON over `/api`.

`src/lib/api-client.ts` is the only frontend module that calls `fetch`. It owns URL construction, JSON encoding, the NDJSON stream reader, and the translation of the error envelope into a typed `ApiError` carrying status, code, and message. A network failure becomes `ApiError(0, 'network_error', ...)`, so a dead backend and a rejected request are handled the same way by callers. `src/lib/api-host.ts` resolves the base: blank in development so Vite proxies `/api`, or `VITE_API_HOST` when the API is hosted elsewhere.

State lives in hand-written hooks rather than a query library. `useAgents` and `useSessions` are each mounted once through a provider so every surface shares one list, and both apply writes optimistically with rollback held per entity rather than per list snapshot. `useChat` holds threads as `Record<sessionId, Message[]>`, consumes the streaming chat route event by event, and rehydrates a reopened conversation from `GET /api/runs?sessionId=` through `lib/run-to-messages.ts`.

Design decision. The API is authoritative for everything persisted; the browser holds presentation and interaction state only. A chat thread is reconstructed from sessions and runs rather than being durable browser state, which is why history survives a reload on a different machine and why a rehydrated thread shows exactly what was recorded.

2.2 API reference

The base path is `/api`. In development the FastAPI application serves it directly on port 4000 and Vite proxies to it from 5173. In the container stack nginx publishes port 8080 and proxies `/api/` to `api:4000` on the internal network, so the browser is always same-origin and never issues a CORS preflight.

All bodies are JSON. **Wire JSON is camelCase and Python identifiers are snake_case**: every schema crossing the boundary inherits `WireModel`, which applies a camelCase alias generator. A field that misses that base class silently reaches the frontend as `undefined` — no error, just empty UI.

Three cross-cutting behaviors apply before any route runs:

- **Body size.** `BodySizeLimitMiddleware` rejects a request whose `Content-Length` exceeds `MAX_BODY_BYTES` (default 262,144) and also counts the streamed body, so a chunked request cannot bypass the header check. A non-numeric `Content-Length` is a 400 with `Malformed Content-Length header`. The middleware sits inside CORS so a 413 still carries `Access-Control-Allow-Origin`.
- **JSON parsing.** `core/http.json_body` reproduces `express.json()` deliberately, because the contract this service replaced was an Express application. A body that is absent or not sent as `application/json` parses as `{}` rather than failing; strict mode requires the first non-whitespace byte to open an object or array; `NaN` and `Infinity` are rejected; and a lone UTF-16 surrogate is rejected at the door rather than raising during response serialization after the write already happened. All of those answer `Malformed JSON body`.
- **String lengths are measured in UTF-16 code units**, matching JavaScript's `String.length` rather than Python's `len()`, so an emoji costs 2 and the frontend's live character

counters agree with the server.

2.2.1 Error envelope

Every failure — without exception — serializes as:

```
{ "error": { "code": "bad_request", "message": "..."} }
```

FastAPI fights this in two places, and both have an explicit handler: request validation failures default to 422 with FastAPI's own body shape, and unmatched routes raise `StarletteHTTPException` whose default body is `{"detail": ...}`.

Status	code	When
400	<code>bad_request</code>	Validation failure, malformed JSON, or a request-validation failure FastAPI would return as 422
404	<code>not_found</code>	Missing resource, or an unmatched method and path — an unsupported method answers 404, not 405, with <code>No route for PUT /api/agents</code>
413	<code>payload_too_large</code>	Body exceeds <code>MAX_BODY_BYTES</code> ; message is always <code>Request body is too large.</code>
502	<code>provider_error</code>	The model provider failed or timed out; message is one of five classified sentences
500	<code>internal_error</code>	Unhandled exception; the public message is always <code>Something went wrong on the server.</code>

The frontend parses this shape in `api-client.ts` and surfaces `message` directly, which is why messages are written as sentences a person can act on.

2.2.2 Health

Method	Path	Response
GET	<code>/api/health</code>	<code>{ "status": "ok", "mode": "mock" "live" }</code>

`mode` reports the configured `LLM_PROVIDER` and nothing more. It does not prove a credential is valid or that a provider call has ever succeeded — with `LLM_PROVIDER=adk_gemini` and an empty key the service still starts cleanly and reports `live`, and the failure appears as a 502 on the first chat request. The sidebar status pill polls this every five minutes.

2.2.3 Agents

An agent is the mutable configuration used by future executions.

Method	Path	Status	Response
GET	<code>/api/agents</code>	200	<code>Agent[]</code> , newest <code>updatedAt</code> first
POST	<code>/api/agents</code>	201 / 400	<code>Agent</code>
GET	<code>/api/agents/{agentId}</code>	200 / 404	<code>Agent</code>
PATCH	<code>/api/agents/{agentId}</code>	200 / 400 / 404	<code>Agent</code>
DELETE	<code>/api/agents/{agentId}</code>	204 / 404	—

Agent

Field	Type and constraints	Ownership
id	string, prefixed <code>agent_</code>	Server-owned
name	string, at most 120 characters	Writable
icon	string, at most 32 characters	Writable
description	string, at most 2,000 characters	Writable
model	string, at most 64 characters, must be a known model id	Writable
systemPrompt	string, at most 20,000 characters	Writable
toolIds	array of known tool ids, no duplicates, no longer than the registered catalog	Writable
status	"active" or "draft"	Writable
createdAt, updatedAt	ISO-8601 timestamps	Server-owned

Write validation runs in a fixed order, and the first failing rule determines the response:

1. The six string fields, in the order `name`, `icon`, `description`, `model`, `systemPrompt`, `status` — type first, then length.
2. `model` against the known model set.
3. `status` against `{"active", "draft"}`.
4. `toolIds` — array, then all strings, then catalog size, then duplicates, then unknown ids.

Representative messages, asserted character for character by the contract tests: `name must be at most 120 characters.`, `Unknown model "x".`, `Unknown status "x".`, `toolIds must not contain duplicates.`, `Unknown tool ids: a, b.`

An absent body is treated as `{}`. A non-object body answers `Request body must be a JSON object`. Keys outside the seven writable fields — including `id`, `createdAt`, and `updatedAt` — are **silently dropped** from a create or patch rather than rejected, because a client that echoes a whole agent back should not have to strip the server-owned fields first.

`POST /api/agents` with an empty body is valid and produces a usable draft: name `New agent`, the default model, empty description and prompt, no tools, status `draft`. Both timestamps come from one clock read, so a new agent's `createdAt` and `updatedAt` are identical rather than microseconds apart. `updatedAt` moves on every patch, including an empty one.

`PATCH` looks the agent up **before** validating the body, so a patch to a deleted agent answers 404 rather than a validation error. Deleting an agent also deletes its sessions and its triggers, and deliberately keeps its runs. Section 3.2 sets out why.

2.2.4 Tools and models

Two read-only catalogs, describing what may be attached to an agent. Both routes read their injected port directly and have no service layer, because there is no business rule between the request and the catalog.

Method	Path	Response
GET	<code>/api/tools</code>	<code>ToolSchema[]</code> — { <code>id</code> , <code>label</code> , <code>description</code> , <code>params</code> }, in registration order
GET	<code>/api/models</code>	<code>ModelInfo[]</code> — { <code>id</code> , <code>label</code> }

Registration order is contract, not incidental: it is the order the frontend tool picker renders. The model catalog currently holds one entry, `gemini-3.1-flash-lite` (Gemini 3.1 Flash Lite), which is also the default model applied to a new agent.

2.2.5 Chat and execution

Two routes execute one complete agent turn. They perform identical work and differ only in how the result reaches the client.

Method	Path	Status	Response
POST	/api/chat/{agentId}/messages	200 / 400 / 404 / 413 / 502	{ message, session? } as JSON
POST	/api/chat/{agentId}/messages/stream	200 / 400 / 404 / 413	application/x-ndjson event stream

The frontend uses the streaming route so tool activity and model text appear as they happen; the JSON route is the simpler contract, and most of the chat contract tests use it; one test covers the stream end to end, asserting that the run is persisted and that its answer matches the done event.

2.2.5.1 Request body

```
{ content: string, retry?: boolean, sessionId?: string }
```

- `content` is required, trimmed, must be non-empty after trimming, and at most 10,000 UTF-16 code units. The trimmed value is what gets measured, validated, and stored.
- Omitting `sessionId` starts a new chat and the response carries the created `session`.
- Later messages send the returned `session.id`. An unknown session is a 404; a session belonging to another agent is a 400 reading `Session "sess_x" does not belong to agent "agent_y"`.
- On every message after the first, `session` is **absent** from the response rather than null — the route sets `response_model_exclude_none`, so clients test `if (response.session)`.
- `retry: true` suppresses the deterministic failure path for that request only. It is request metadata, never server state, so identical messages from different clients stay independent.

Validation order is content type, then blankness, then length, then `retry`, then `sessionId`, so a body breaking two rules reports the earlier one. `retry` is type-checked explicitly rather than coerced: `retry: "yes"` is a 400, not `true`.

2.2.5.2 Response shapes `MessageResponse` carries `id`, `role: "assistant"`, `content`, `toolCalls[]`, `model`, `latencyMs`, `status: "done" | "error"`, and `createdAt`. The turn's status is `error` when any tool call failed, even though the answer itself is present.

Each `ToolCall` carries `id`, `toolId`, `args`, `durationMs`, `status: "ok" | "error"`, plus `result` on success and error on failure. The unused key is **absent**, not null, because the trace rail renders "no result" differently from a result that is literally `null`.

2.2.5.3 Streaming events Newline-delimited JSON, one object per line, sent with `Cache-Control: no-cache` and `X-Accel-Buffering: no`; nginx sets `proxy_buffering off` on `/api/` so the stack does not hold events back.

type	Payload	Meaning
session	session	Emitted first, and only on the request that created a session
textDelta	text	A fragment of model text
toolStarted	call with <code>status: "running"</code> and <code>durationMs: 0</code>	A tool invocation began

type	Payload	Meaning
toolFinished	callId, result, error, durationMs, status	That invocation completed or failed
done	message	The same MessageResponse the JSON route returns
error	error with code and message	The turn failed after headers were already sent

Validation and agent lookup happen **before** the response starts, so a bad request still gets a normal status code and the error envelope. Once the stream has opened the status is already 200, which is why a later failure arrives as an in-band **error** event; the client turns it back into an **ApiError**. A stream that ends without a **done** event is treated as a failure by the client rather than as a successful empty turn.

2.2.5.4 The deterministic failure path `MockLLMProvider` exists so the full flow — tool calls, trace, failure, retry, persisted run — is reachable with no API key, and so the contract tests are repeatable. Its behavior is contract:

- It calls at most **two** tools, taken in the agent's configured order, and skips any tool with no fixture.
- Content matching `/\bfail/i` is a word-boundary prefix match: **fail**, **failure**, and **failing** trigger it; **unfail** does not. When it triggers and the agent has at least one fixture-backed tool, the **last attempted** call fails with **connection refused after 800ms** and a `durationMs` of 812, and the answer explains the failure.
- `retry: true` suppresses that, so the same message succeeds on the next attempt.
- `latencyMs` is the sum of the tool durations plus a fixed 180 ms of model time.
- It emits no `textDelta` events; over the streaming route a mock turn shows tool events followed by **done**. Live Gemini streams text, so the two look different on screen by design.

A retry appends a **new** run and leaves the failed one in place. The live thread replaces the failed turn with its retry, so a rehydrated thread shows both attempts where the live one showed only the success. That is deliberate: the run log is an audit trail, and a failure is never rewritten out of it.

2.2.6 Sessions

A session is the identity and title of one chat thread. It deliberately carries no agent name, icon, or model — those are snapshotted on runs, because they are an audit concern, whereas a session is navigation and must reflect a rename immediately.

Method	Path	Status	Response
GET	<code>/api/sessions?limit=</code>	200	<code>Session[]</code> , newest activity first; limit 1–200, default 50
PATCH	<code>/api/sessions/{sessionId}</code>	200 / 400 / 404	<code>Session</code>
DELETE	<code>/api/sessions/{sessionId}</code>	204 / 404	—

Session: `id` (prefixed `sess_`), `agentId`, `title`, `createdAt`, `updatedAt`.

A session is created only by the first message of a chat and is titled from that message: whitespace collapsed, cut at a word boundary near 60 characters with an ellipsis, hard-cut

for a single long word, and the placeholder `New chat` for a message with no visible content. Its `updatedAt` is touched by every message, which is what orders the sidebar.

Rename accepts `{ title: string }`. The title is type-checked, capped at 120 UTF-16 code units, then trimmed; a missing or blank title answers `title is required`. Other keys are dropped silently.

Validation runs **before** the lookup here, which is the opposite of `PATCH /api/agents/{id}`. The two differ only when the id is unknown *and* the body is invalid: sessions answer 400 and agents answer 404. Sessions are renamed from a list the client already holds, so a bad body is the likelier mistake and naming it is the more useful answer. This asymmetry is recorded rather than aligned.

Deleting a session deletes its runs. That cascade lives in `SessionService`, not in a foreign key, so both storage backends behave identically.

2.2.7 Triggers

A trigger stores an agent, a schedule, and one literal message sent verbatim on every firing.

Method	Path	Status	Response
GET	<code>/api/triggers?agentId=</code>	200	<code>Trigger[]</code> , newest <code>updatedAt</code> first; limit 1–200, default 50
POST	<code>/api/triggers</code>	201 / 400 / 404	<code>Trigger</code> ; 404 when <code>agentId</code> is unknown
GET	<code>/api/triggers/{triggerId}</code>	200 / 404	<code>Trigger</code>
PATCH	<code>/api/triggers/{triggerId}</code>	200 / 400 / 404	<code>Trigger</code>
DELETE	<code>/api/triggers/{triggerId}</code>	204 / 404	—; keeps historical runs
POST	<code>/api/triggers/{triggerId}/run</code>	200 / 404	<code>Run</code> — fire once, now

Trigger

Field	Type and constraints
<code>id</code>	Server-owned, prefixed <code>trg_</code>
<code>agentId</code>	Non-empty string; required on create, and the agent must exist
<code>name</code>	String, at most 120 characters; defaults to <code>New trigger</code>
<code>message</code>	String, at most 20,000 characters; must be non-blank on create
<code>kind</code>	"interval" or "daily"; required on create
<code>intervalMinutes</code>	Integer at least 1 for <code>kind="interval"</code> ; null otherwise. <code>true</code> is rejected, not coerced
<code>timeOfDay</code>	HH:MM in 24-hour form for <code>kind="daily"</code> ; null otherwise
<code>weekdays</code>	Integers 0–6 with Monday as 0, no duplicates; empty means every day, and an interval trigger always stores empty
<code>timezone</code>	Valid IANA zone name, at most 64 characters; defaults to UTC
<code>enabled</code>	Boolean; defaults to true
<code>nextRunAt</code>	Server-owned; null while disabled
<code>lastRunAt</code> , <code>lastStatus</code> , <code>lastRunId</code>	Server-owned outcome of the most recent firing
<code>createdAt</code> , <code>updatedAt</code>	Server-owned

A patch is validated **against the current trigger**, not against the body alone, because "an interval trigger needs `intervalMinutes`" is a question about the trigger's effective kind. Switching

`kind` to `interval` without sending `intervalMinutes` therefore succeeds only when the stored trigger already has a usable interval. Schedule normalization then blanks the fields the other kind owns, so an interval trigger can never keep a stale `timeOfDay` and read back a schedule that is not the one that fires.

`nextRunAt` is recomputed only when the body actually contains a scheduling key — `kind`, `intervalMinutes`, `timeOfDay`, `weekdays`, `timezone`, or `enabled`. Testing the normalized result instead would recompute on every patch, which for an interval trigger would push the next firing out by a full interval every time somebody renamed it.

Daily schedules resolve both daylight-saving edges explicitly. A wall-clock time that occurs twice picks the first pass; a time that does not exist at all is walked forward to the first instant that does.

Server-owned fields are dropped silently from create and patch bodies, the same way agents drop `id` and `createdAt`. `PATCH` looks the trigger up before validating, so a patch to a deleted trigger answers 404.

Fire-now works whether or not the trigger is enabled and whether or not the scheduler is running — firing before turning a schedule loose is the main reason it exists — and it deliberately does **not** reschedule. A triggered run, however it was started, has `sessionId`: `null` and its `triggerId` set.

2.2.8 Runs

A run is the historical record of one completed execution. Runs are appended, never updated; explicit deletion of a run's agent history, its session, or the run itself is the only way one disappears.

Method	Path	Status	Response
GET	/api/runs?agentId=&limit=	200	Run[] for one agent
GET	/api/runs?sessionId=&limit=	200	Run[] for one chat
GET	/api/runs?triggerId=&limit=	200	Run[] for one trigger
GET	/api/runs/{runId}	200 / 404	Run
DELETE	/api/runs?agentId=	204 / 400	Deletes every run for one agent

`limit` is 1–200, default 50, and results are newest first. At most one of `agentId`, `sessionId`, and `triggerId` may be supplied; omitting all three lists runs across every agent. The `agentId` plus `sessionId` case answers `Pass agentId` or `sessionId`, not `both`. verbatim, because that message predates triggers; any combination involving `triggerId` answers `Pass at most one of agentId, sessionId, triggerId`.

`DELETE` is deliberately asymmetric with `GET`: a missing `agentId` on the read means "every agent", but on a delete that default would erase the entire history, so it answers `agentId is required`.

Run: `id`, `agentId`, `agentName`, `model`, `systemPrompt`, `userMessage`, `answer`, `status`, `error`, `latencyMs`, `sessionId`, `triggerId`, `createdAt`, and `toolCalls[]`.

Each `RunToolCall` carries `id`, `seq`, `toolId`, `args`, `result`, `error`, `durationMs`, and `status`. `seq` is the invocation order, assigned when the call started rather than when it finished, so a provider that runs tool calls concurrently still persists them in the order the model asked for them.

`agentName`, `model`, and `systemPrompt` are execution-time snapshots, which is what makes this endpoint an audit trail rather than a join against current configuration.

2.3 Data model

2.3.1 Entities and relationships

The persistent domain is five tables: agents, chat sessions, triggers, runs, and the tool calls belonging to each run. Agents, sessions, and triggers are mutable operational resources. Runs and run tool calls are immutable history.

The split matters because the two kinds need opposite lifecycle rules. A user must be able to rename or delete current configuration without silently changing the evidence of what the system already did. At the same time, deleting a chat thread is expected to remove the turns that make up that thread.

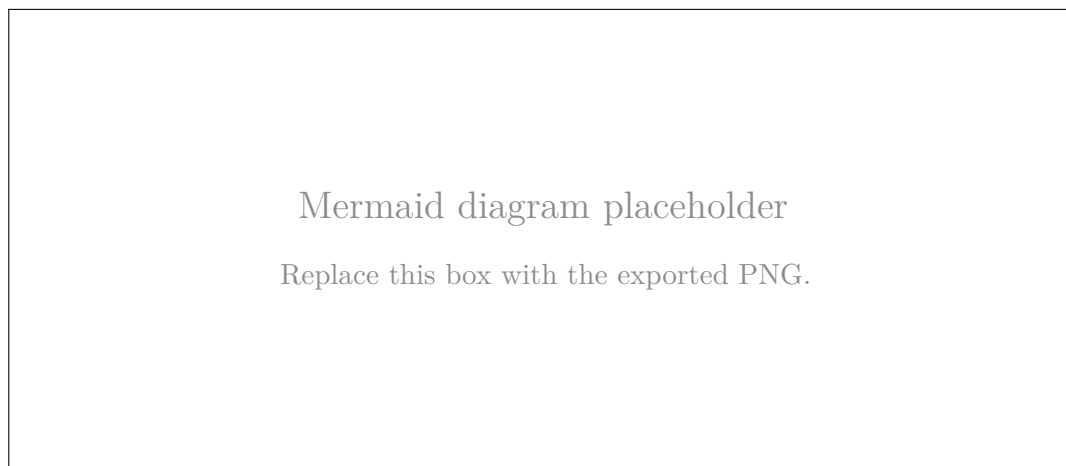


Figure 9: Persistent entities and relationships

Design decisions.

- **Runs snapshot agent_name, model, and system_prompt** because agents are mutable, and resolving those through the current agent would make an edit retroactively rewrite history.
- **No table has a foreign key to agents.** That is what allows the application to preserve runs while deleting the sessions and triggers that cannot function without the agent. A database-level cascade would make the audit trail impossible to keep.
- **session_id and trigger_id on runs are both nullable** and are what distinguish the two execution contexts. A chat run has a session and no trigger; a triggered run has the reverse.
- **run_tool_calls.run_id is a real cascading foreign key**, because a tool call has no meaning or lifecycle independent of its run.
- **PostgreSQL is the durable store; the in-memory store is a full peer.** Both satisfy the same repository protocols, including ordering and copy semantics, and one shared contract test suite runs against both. That is what keeps `uv run pytest` honest without a database.

2.3.2 Deletion behavior

Operation	Deleted	Preserved	Why
Delete an agent	The agent, its chat sessions, its triggers	Its runs and their tool calls	History outlives mutable configuration. A session or trigger pointing at a missing agent can never be continued or fired, so leaving it would only put a permanently broken row in a list

Operation	Deleted	Preserved	Why
Delete a trigger	The trigger	Runs it fired	Trigger activity is historical evidence, on the same terms as an agent's
Delete a session	The session and its runs	The agent and unrelated runs	The thread and its turns are one thing to a user, and an orphaned run is unreachable from the UI
Delete a run	The run and its tool calls	Everything else	Tool calls are owned by the run

The first three rules live in the owning application service, not in the database, so the memory and PostgreSQL backends behave identically. Only the last is a database cascade.

2.3.3 Physical schema

Table	Columns	Indexes and constraints
agents	id PK, name, icon, description, model, system_prompt, tool_ids JSONB, status, created_at, updated_at	agents_updated_at_idx on updated_at DESC
runs	id PK, agent_id, agent_name, model, system_prompt, user_message, answer, status, error, latency_ms, session_id, trigger_id, created_at	Indexes on created_at DESC and on agent, session, and trigger id each with created_at DESC; no FK to agents, sessions, or triggers
run_tool_calls	id PK, run_id, seq, tool_id, args JSONB, result JSONB, error, duration_ms DOUBLE PRECISION, status	run_id REFERENCES runs(id) ON DELETE CASCADE; index on (run_id, seq)
chat_sessions	id PK, agent_id, title, created_at, updated_at	Index on updated_at DESC; no FK to agents
triggers	id PK, agent_id, name, message, kind, interval_minutes, time_of_day, weekdays JSONB, timezone, enabled, next_run_at, last_run_at, last_status, last_run_id, created_at, updated_at	Partial index on next_run_at WHERE enabled, plus indexes on agent_id and updated_at DESC; no FK to agents

Two column choices are worth calling out. The due index on `triggers` is **partial** because a disabled trigger has `next_run_at = NULL`, so the index the scheduler polls on every tick stays proportional to the enabled set rather than the whole table. And `run_tool_calls.duration_ms` is `DOUBLE PRECISION` because an in-process tool answers in tens of microseconds, and integer truncation reported every one of them as 0 ms — indistinguishable from a call that never ran.

JSONB is decoded to Python objects once, in the connection initializer, so no caller has to remember to parse `tool_ids` out of a string.

2.3.4 Startup schema management and backfill

With `STORE_BACKEND=postgres`, the lifespan opens the pool, applies `app/core/schema.sql`, and seeds the four sample agents **only when the agents table is empty**, logging `postgres ready; seeded 4 agents on first boot and seeded 0 agents afterwards`. The guard is what stops a restart from resurrecting an agent someone deleted.

Every statement in the schema file is idempotent. Two later changes could not be expressed with `CREATE TABLE IF NOT EXISTS` alone and have their own guarded statements: `runs.trigger_id` is added with `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`, and the `duration_ms` widening runs inside a `DO` block that checks `information_schema` first.

This is startup schema management, not a migration framework. It handles creation and the two recorded widenings; any further change against a populated database is a manual job. Alembic is the intended answer once the schema changes under real data.

The lifespan then runs `backfill_sessions`, a one-time migration for run history written before sessions existed. It reads runs whose `session_id` is still null, oldest first, groups them by agent, creates one session per agent titled by the same deterministic truncation, and assigns the runs to it. It is a no-op on every boot after the first and on an empty memory store, and a failure is logged and swallowed rather than blocking startup — booting with some history un-migrated is recoverable on the next boot, while failing to boot is a total outage.

2.4 Extension interfaces

The two extension points are deliberately narrower than the application. Adding a tool or a model provider should not require touching execution orchestration, HTTP routes, agent management, or persistence.

2.4.1 Tool boundary

A tool is one file holding its identity, its schema, and its behavior. The registry owns the catalog; nothing else in the codebase knows which tools exist.

```
class ToolParam(WireModel):
    name: str
    type: Literal["string", "number", "boolean"]
    required: bool = False
    description: str

class ToolSchema(WireModel):
    id: str
    label: str
    description: str
    params: list[ToolParam]

class ToolResult(WireModel):
    ok: bool
    value: Any | None = None
    error: str | None = None

@runtime_checkable
class Tool(Protocol):
    schema: ClassVar[ToolSchema]

    async def execute(self, **kwargs: Any) -> ToolResult: ...
```

ToolRegistry exposes exactly what the rest of the system needs:

Method	Purpose
<code>list()</code>	Schemas for GET <code>/api/tools</code> , in registration order
<code>get(tool_id)</code>	One tool, or None
<code>resolve(ids)</code>	Stored ids into tools in the requested order; unknown ids raise a 400
<code>known_ids()</code>	The allowed id set, so agent validation never imports the catalog
<code>invoke(tool_id, **kwargs)</code>	Registry-level invocation entry point

Design decisions.

- **Expected failures are values, not exceptions.** A bad argument or a refused connection returns `ToolResult(ok=False, error=...)`, so the model observes the failure and explains it instead of the turn collapsing. Only a genuine infrastructure fault should propagate, and even then the ADK adapter catches it, records it as a failed call, and returns the error to the model.
- **`known_ids()` exists so agents/ can validate without importing `tools.impls`.** That single method is what keeps the agent module free of the catalog, and `tests/test_structure.py` enforces it.
- **The ADK bridge synthesizes a real signature.** Our protocol is `execute(**kwargs)`, which advertises no parameters; handed to ADK unchanged, every tool would look argument-less and the model would never pass one. `adk_adapter.py` builds an `inspect.Signature` from `ToolSchema`, writes the docstring ADK turns into the function declaration, times each call to hundredths of a millisecond, and claims each call's position in the trace **before** awaiting it, so concurrent calls are ordered by when the model asked rather than by which finished first.

Built-in tools, in registration order:

id	Behavior	Notable guard
<code>current_time</code>	Current time in an optional IANA zone, defaulting to UTC	An unknown zone is a failed call, not a 500
<code>http_request</code>	Fetches a URL and returns <code>{status, latencyMs, body}</code>	See below
<code>calculator</code>	Evaluates an arithmetic expression	Parses to an AST and walks it — <code>eval()</code> on a model-supplied string is remote code execution. Exponents are bounded, and non-finite results are rejected before they can reach JSON
<code>knowledge_search</code>	Searches a small fixed corpus, scoring by normalized term overlap	Deterministic and explainable; a real deployment would swap it for a vector store

`http_request` is the one tool reachable by prompt injection, since the model chooses the URL from user text. It allows only `http/https` and only `GET/HEAD`, resolves the hostname and requires **every** resolved address to be publicly routable (`is_global`, rather than a list of negative flags that would let carrier-grade NAT ranges through), disables redirects, bounds the request by `TOOL_HTTP_TIMEOUT_MS`, and reads the body in chunks so it hangs up at 64 KB rather than buffering a large file or a compression bomb into the process. The residual gap is recorded rather than hidden: the address is resolved and checked, then `httpx` resolves it again to connect, so a DNS record changing between the two could still slip through. Closing that needs connection-level pinning, which is beyond this proof of concept.

2.4.2 LLM provider boundary

The provider port represents one complete agent turn, including any model-directed tool loop — not a single text completion.

```
@dataclass
class RunSpec:
    agent_id: str
    name: str
    model: str
    system_prompt: str
    tools: list[Tool]
```

```

user_message: str
retry: bool
session_id: str | None = None
trigger_id: str | None = None
timezone: str | None = None

RunEvent = ToolCallStarted | ToolCallFinished | TextDelta | TurnFinished

class LLMPProvider(Protocol):
    def models(self) -> list[ModelInfo]: ...

    def run(self, spec: RunSpec) -> AsyncIterator[RunEvent]: ...

```

Event	Data	Meaning
ToolCallStarted	call_id, tool_id, args	A tool invocation began
ToolCallFinished	call_id, result, error, duration_ms	That invocation completed or failed
TextDelta	text	Incremental model text, forwarded by the streaming route
TurnFinished	text, model, latency_ms	The authoritative final result; exactly one per turn

Design decisions.

- **Normalizing here is what makes the mock and the live provider interchangeable**, and what lets the entire contract suite run without an API key. The cost is that every adapter must preserve call pairing, invocation order, result and error shape, model identity, and latency while hiding whatever its SDK emits beyond those four events.
- **RunSpec.timezone exists because the container clock is UTC** but a trigger stores the zone it was configured in. The ADK adapter turns it into an instruction paragraph, so a time-aware tool call with no explicit argument defaults to the zone the user actually picked rather than to the container's.
- **The optional SDK import is deferred into the methods that use it.** A module-scope import would make google-adk mandatory just to boot or test the service, and `tests/test_structure.py` imports every module.

AdkGeminiProvider specifics. It drives ADK's Runner in SSE streaming mode and forwards text as `TextDelta`, dropping parts flagged `thought=True` so the model's internal reasoning never reaches the user. Tool events are replayed from the recorder **after** the runner completes rather than correlated with ADK's own call ids, because ADK does not report durations and `durationMs` is contract. The agent's ADK name is its `id`, since ADK requires a Python identifier and `Support Bot` is not one; the display name reaches the model through an instruction paragraph appended after the system prompt, which also overrides ADK's own identity line. Any SDK exception becomes a `ProviderError` carrying a classified sentence.

2.5 Deployment guide

2.5.1 Run the container stack

Prerequisite: Docker Desktop or another Docker Compose-compatible engine. This path needs no local Python and no local Node.

1. Move to the deployment directory:

```
Set-Location .\AgentPlatform-BackEnd\deployment
```

2. Build and start:

```
docker compose up --build
```

A fresh clone needs no configuration file: every setting has a default in `docker-compose.yml`, so the stack comes up against PostgreSQL with the deterministic mock provider and no credentials. Open `http://localhost:8080`.

3. Confirm what should have happened:

- `db` becomes healthy once `pg_isready` succeeds.
- `api` starts only after that, applies `app/core/schema.sql`, and listens on 4000 inside the network. It publishes no host port — the only way in is through nginx.
- `web` starts only after `GET /api/health` succeeds, serves the built client, and proxies `/api/` to `api:4000` on the same origin.
- First boot logs `postgres ready; seeded 4 agents`; later boots log `seeded 0 agents`.

To change anything, copy the template and edit it:

```
Copy-Item .\env.example .\env
```

Note that the template ships with `WEB_PORT=8090`, so once you copy it the UI moves to `http://localhost:8090` until you change that line. Compose reads `.env` from this directory automatically because it is the directory holding `docker-compose.yml`. The file holds credentials, is gitignored, and must never be committed.

The `pgdata` named volume survives `docker compose down`. To discard the database and start empty:

```
docker compose down -v
```

That permanently deletes every agent, session, trigger, run, and tool call in the volume.

2.5.2 Configuration reference

`app/config.py` is the only module that reads the environment. It also reads a `.env` file in the server directory, which is worth knowing during local development: a stale local `.env` overrides the code defaults for anything run from that directory, including the test suite.

Variable	Code default	In the base Compose stack
PORT	4000	Declared but never read. The listening port comes from the container command and from the <code>--port</code> flag locally
CORS_ORIGIN	<code>http://localhost:5173</code>	Set to <code>http://localhost:8080</code> ; a fallback only, since nginx makes the browser same-origin
STORE_BACKEND	<code>memory</code>	Fixed to <code>postgres</code> — it describes the topology, so setting it in <code>.env</code> does nothing
DATABASE_URL	<code>postgresql://app:app@db:5432/agents</code>	Built by Compose from <code>POSTGRES_PASSWORD</code>
LLM_PROVIDER	<code>mock</code>	Forwarded from <code>.env</code> ; <code>mock</code> or <code>adk_gemini</code>

Variable	Code default	In the base Compose stack
GEMINI_API_KEY	empty	Forwarded from <code>.env</code>
TOOL_HTTP_TIMEOUT_MS	5000	Forwarded from <code>.env</code>
LOG_PAYLOAD_MAX_BYTES	32768	Forwarded from <code>.env</code>
MAX_BODY_BYTES	262144	Code default; not exposed by the base Compose file
TRIGGERS_ENABLED	true	Code default; not exposed by the base Compose file
TRIGGER_TICK_SECONDS	30	Code default; not exposed by the base Compose file
TRIGGER_MAX_PERIOD_SECONDS	20	Code default; not exposed by the base Compose file

Only variables named in the `api.environment` block of `docker-compose.yml` reach the API container. There is no `env_file`, and that is deliberate: the block is the complete, readable inventory of what the container receives, which is also how the Cloudflare tunnel token stays out of the API's environment. Adding a setting to `app/config.py` means adding a line there too. Putting a name in `.env` alone does nothing for an unlisted setting.

Compose itself also reads:

Variable	Default	Scope
WEB_PORT	8080 in the compose file, 8090 in <code>.env.example</code>	Host port nginx is published on
WEB_BIND	0.0.0.0	Host interface that port binds to
POSTGRES_PASSWORD	app	Database initialization, and the generated <code>DATABASE_URL</code>
PUBLIC_ORIGIN	none	Required by <code>docker-compose.prod.yml</code>
CLOUDFLARE_TUNNEL_TOKEN	none	Required by <code>docker-compose.prod.yml</code>

PostgreSQL applies `POSTGRES_PASSWORD` at `initdb` only. Changing it against an existing `pgdata` volume has no effect, so set it before the first boot.

2.5.3 Run against live Gemini

1. Set both values in `AgentPlatform-BackEnd/deployment/.env`:

```
LLM_PROVIDER=adk_gemini
GEMINI_API_KEY=your-key
```

2. Rebuild and restart the API, so the image carries the `adk` extra:

```
docker compose up -d --build api
```

3. Verify `GET /api/health` reports `"mode": "live"`, then send a chat message.

The health response reports configuration only. An empty or invalid key is discovered on the first provider call, which answers `502 provider_error`.

The platform has no authentication and no rate limiting. Every route is open, including `DELETE /api/agents/{id}`. An open instance running `adk_gemini` is a free LLM proxy

on your key. Keep a publicly reachable instance on `mock`, or put an access-control layer — the repository's production overlay uses a Cloudflare Tunnel with Cloudflare Access — in front of it first, and close the published host port with `WEB_BIND=127.0.0.1` so the tunnel is the only way in.

2.5.4 Common deployment problems

Symptom or task	Cause and action
The UI is not on 8080	You copied <code>.env.example</code> , which sets <code>WEB_PORT=8090</code> . Change the line or use that port
Port already in use	Change <code>WEB_PORT</code> in <code>.env</code>
A changed database password has no effect	PostgreSQL kept the password stored in the existing <code>pgdata</code> volume. Migrate the credential, or recreate the volume if the data is disposable
Live mode starts but chat returns <code>provider_error</code>	Check <code>GEMINI_API_KEY</code> ; startup never validates it
A trigger setting in <code>.env</code> has no effect	The base Compose file does not forward <code>TRIGGERS_ENABLED</code> , <code>TRIGGER_TICK_SECONDS</code> , or <code>TRIGGER_MAX_PER_TICK</code> . Add an explicit <code>environment:</code> entry
<code>STORE_BACKEND=memory</code> in <code>.env</code> is ignored	Intentional. The base file fixes it to <code>postgres</code> , because memory would detach the API from the database it just waited for
Data vanished after a reset	<code>docker compose down -v</code> deletes the named volume; <code>docker compose down</code> alone preserves it

2.5.5 Run locally without containers

The two projects have separate toolchains and no root workspace; run each command from the directory named.

Backend, from `AgentPlatform-BackEnd/server` — Python 3.14, managed with `uv`:

```
uv sync
$env:STORE_BACKEND = "memory"
$env:LLM_PROVIDER = "mock"
uv run uvicorn app.main:app --port 4000 --reload
```

Memory and mock are already the code defaults; setting them explicitly states the deterministic intent and overrides any stale local `.env`. **Port 4000 is load-bearing** — the Vite proxy, the nginx configuration, and the Compose healthcheck all target it.

Frontend, from `AgentPlatform-FrontEnd/client` — Node 24:

```
npm ci
npm run dev
```

Vite listens on 5173 and proxies `/api` to `http://localhost:4000`, which is why `api-host.ts` resolves to a blank base in development: requests stay same-origin and no preflight happens.

2.6 Developer guide

2.6.1 Add a tool

1. Create a module under `app/modules/tools/impls/`, for example `weather.py`.

2. Implement `Tool`: declare `schema: ClassVar[ToolSchema]`, implement `async def execute(self, **kwargs) -> ToolResult`, and return `ToolResult(ok=False, error=...)` for expected argument or business failures. Let genuine infrastructure faults propagate; the provider adapter records them as a failed call rather than a 500.
3. Register it in `default_tools()` in `app/modules/tools/registry.py`. Its position in that list is its position in `GET /api/tools` and in the frontend picker.
4. If deterministic demonstrations should exercise it, add a fixture to `TOOL_FIXTURES` in `app/modules/llm/providers/mock.py`. Without one the mock skips the tool — a deliberate degradation rather than a break — while the live provider still receives it normally.
5. Add tests for schema identity and parameter shape, a successful execution, an expected failure as `ToolResult(ok=False, ...)`, and registry resolution if relevant.
6. Run:

```
uv run pytest tests/unit/test_tools.py tests/unit/test_tool_registry.py
uv run ruff check .
```

Expected result. The schema appears in `GET /api/tools`, agents accept its id in `toolIds`, and the live provider can call it — with no change to `ExecutionService`, any router, or agent validation.

2.6.2 Add an LLM provider

1. Create a module under `app/modules/llm/providers/`.
2. Implement `LLMProvider`: `models()` and `run(spec)`.
3. Translate the SDK's native events into only `ToolCallStarted`, `ToolCallFinished`, `TextDelta`, and `TurnFinished`. Preserve call pairing, invocation order, result and error shape, model identity, and latency, and emit exactly one `TurnFinished`.
4. Convert SDK and upstream failures into `ProviderError` so the API answers 502 rather than 500, and put the classification in `llm/failures.py` rather than interpolating vendor text into a message a user will read.
5. Keep an optional SDK import inside the method that uses it. A module-scope import would make the SDK mandatory to boot or test the service, and `tests/test_structure.py` imports everything.
6. Select it in `get_llm_provider()` in `app/container.py` and add its literal to `Settings.llm_provider` in `app/config.py`. Nowhere else.
7. If it changes the model catalog, update `app/modules/llm/catalog.py` and the frontend's `src/config/models.ts` together — the frontend keeps its own copy for labels, and the two must agree or an agent will show a raw model id.
8. Add tests for model listing, a successful event sequence, tool-call translation, streaming text, and a provider failure.
9. Run:

```
uv run pytest tests/unit
uv run ruff check .
```

Expected result. Switching LLM_PROVIDER changes model execution with no change to ExecutionService, the routes, agent validation, or run persistence.

2.6.3 Add a storage backend

Implement every repository Protocol that applies — AgentRepository, RunRepository, SessionRepository, TriggerRepository — under the owning module's repositories/ package, and select it only in app/container.py. Run the shared repository contract suite against the new implementation: ordering, copy semantics, and the deletion rules in section 3.2 are part of the contract, not of any one backend.

2.6.4 Verification

Backend, from AgentPlatform-BackEnd/server:

```
uv run pytest
uv run ruff check .
```

The suite needs no database. PostgreSQL-backed tests skip unless TEST_DATABASE_URL points at a **throwaway** database — TEST_DATABASE_URL, never DATABASE_URL, because the fixtures drop and truncate the tables, and keeping the two names separate is what stops a stray test run from erasing real history.

```
uv sync --extra adk
$env:TEST_DATABASE_URL = "postgresql://app@app@localhost:5432/agents"
uv run pytest
```

Frontend, from AgentPlatform-FrontEnd/client:

```
npm test
npm run typecheck
npm run lint
npm run build
```

The backend suite is organized in four layers, which is also the order to reach for when something breaks:

Layer	What it pins
tests/contract/	Status codes, exact validation messages, camelCase wire shapes, and the mock's fixtures
tests/unit/	Validation order, the event translator, the scheduler's schedule maths, tool behavior, provider adapters
tests/repositories/	One shared contract suite run against both the memory and PostgreSQL implementations
tests/test_structure.py	The import boundaries the design rests on

2.6.5 Extension checklist

Before considering either extension complete:

1. Confirm GET /api/tools or GET /api/models exposes the new capability.
2. Create or update an agent that selects it.
3. Run a successful chat turn and inspect both the returned tool calls and the persisted run.

4. Exercise one expected failure and confirm it uses the documented error envelope or the failed tool-call shape, not a 500.
5. Run the full backend suite and the linter.

The extension is correctly isolated when no route, agent-management rule, or execution step needs provider-specific or tool-specific branching.

2.7 Known limitations

Stated plainly, because they are consequences of decisions above rather than oversights.

- **No authentication, authorization, or rate limiting.** Every route is open. This is the single largest gap before any real exposure.
- **A turn carries no conversation history.** `RunSpec` holds one `user_message`, `ExecutionService` only ever appends to the run repository and never reads from it, and `AdkGeminiProvider` builds a fresh session service per turn. A session therefore groups and replays runs for the *reader*; the model itself sees only the system prompt and the current message. Adding memory means loading the session's runs in `_spec()` and extending the port to accept them — the data is already stored and ordered.
- **The scheduler assumes one active instance.** The compare-and-swap claim prevents a double-firing, but scheduling still runs inside the API process. Scaling horizontally means moving it to a singleton worker or a managed scheduler.
- **Seeding can race across replicas.** `seed_agents` reads a count and inserts in one transaction, but at READ COMMITTED two replicas starting against an empty database can both read zero; the loser fails on a unique violation during startup. An advisory lock or `ON CONFLICT DO NOTHING` would close it.
- **Startup schema management is not migrations.** Any schema change against a populated database is currently a manual job.
- **No retention on runs and run_tool_calls.** They grow without bound, and there is no backup or restore path for the `pgdata` volume.
- **The `http_request` SSRF guard has a resolve-then-connect gap.** Documented in section 4.1; closing it needs connection-level address pinning.
- **Credentials are plain environment variables,** visible to `docker inspect` and to any process inside the container.
- **Client IPs are the proxy's.** The API does not read `X-Forwarded-For`, so logs attribute every request to nginx or the tunnel.